

# Tugas 6: Tugas Praktikum Mandiri 06

## Klasifikasi Status Persetujuan Pinjaman Menggunakan Support Vector Machine (SVM)

Revani – 0110224111 <sup>1</sup>

<sup>1</sup> Teknik Informatika, STT Terpadu Nurul Fikri, Depok

\*E-mail: [0110224111@student.nurulfikri.ac.id](mailto:0110224111@student.nurulfikri.ac.id)

**Abstract.** Pada tugas praktikum mandiri 6 ini, dilakukan penerapan algoritma *Support Vector Machine* (SVM) untuk melakukan klasifikasi terhadap data calon peminjam dari *Simple Loan Classification Dataset* yang diambil dari Kaggle. Dataset ini berisi informasi tentang data demografis dan finansial seseorang, seperti jenis kelamin, usia, pekerjaan, pendidikan, status pernikahan, penghasilan, skor kredit, dan status pinjaman apakah disetujui (*Approved*) atau ditolak (*Denied*). Tujuan dari tugas ini adalah untuk membangun model prediksi yang dapat mempelajari pola dari data peminjam dan menentukan apakah pengajuan pinjaman akan diterima atau tidak. Selain membangun model, dilakukan juga evaluasi model, analisis akurasi, serta visualisasi hasil menggunakan grafik 2D dan 3D untuk menggambarkan hubungan antar variabel.

### 1. Penjelasan Hasil Praktikum Mandiri

Pada praktikum ini langkah pertama yang dilakukan adalah menghubungkan Google Colab dengan Google Drive agar file dataset bisa diakses langsung dari direktori penyimpanan. Setelah itu, library yang diperlukan seperti *pandas*, *numpy*, *matplotlib*, *seaborn*, dan library dari *scikit-learn* diimpor agar seluruh proses analisis dan pemodelan bisa dilakukan. Dataset *loan.csv* kemudian dibaca menggunakan fungsi *pd.read\_csv()* dan ditampilkan sebagian untuk memastikan data berhasil dimuat.

Dari hasil pengecekan dengan *df.info()* dan *df.describe()*, diketahui bahwa dataset memiliki kolom-kolom seperti *gender*, *age*, *occupation*, *education\_level*, *marital\_status*, *income*, *credit\_score*, dan *loan\_status*. Kolom *loan\_status* digunakan sebagai target atau variabel dependen yang akan diprediksi, sementara kolom lainnya digunakan sebagai fitur atau variabel independen. Setelah data bersih dan tidak ada nilai kosong, dataset dibagi menjadi dua bagian, yaitu data *training* sebesar 80% dan data *testing* sebesar 20%.

Tahap selanjutnya dilakukan *preprocessing*, di mana data numerik distandarisasi menggunakan *StandardScaler()*, sementara data kategorikal dikonversi menjadi bentuk numerik menggunakan *OneHotEncoder()*. Setelah proses ini, data sudah siap untuk dilatih menggunakan model SVM dengan kernel linear. Model kemudian dilatih dengan fungsi *fit()* menggunakan data training yang sudah diproses.

Hasil pelatihan menunjukkan bahwa model SVM berhasil dibuat tanpa error. Proses evaluasi model dilakukan menggunakan data testing dengan menghitung nilai akurasi, *classification report*, dan *confusion matrix*. Hasil evaluasi menunjukkan bahwa model memiliki tingkat akurasi yang cukup baik dalam memprediksi apakah pinjaman disetujui atau tidak. Visualisasi hasil evaluasi juga dibuat menggunakan *heatmap* untuk memudahkan interpretasi hasil prediksi.

Pada bagian visualisasi, dibuat dua jenis grafik. Grafik 2D menampilkan hubungan antara pendapatan (*income*) dan skor kredit (*credit\_score*) dengan pewarnaan berdasarkan usia (*age*). Dari hasil grafik terlihat bahwa semakin tinggi pendapatan dan skor kredit, biasanya individu tersebut memiliki peluang lebih besar untuk disetujui pinjamannya. Sedangkan grafik 3D menampilkan hubungan antara pendapatan, skor kredit, dan usia, dengan pewarnaan berdasarkan status pinjaman. Dari grafik ini terlihat bahwa kelompok dengan skor kredit dan penghasilan yang tinggi serta usia produktif cenderung mendapatkan status “Disetujui”. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum mandiri 06.

## 2. Penjelasan Kode Program

Langkah awal yang dilakukan adalah menggunakan `from google.colab import drive` untuk menghubungkan Google Colab dengan akun Google Drive agar dataset yang tersimpan bisa dibuka. Setelah drive terhubung, digunakan path direktori untuk mengakses file *loan.csv* seperti yang ditunjukkan pada Gambar 1.

▼ Menghubungkan ke Drive

```
[58] ✓ 1s from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[59] ✓ 0s path = "/content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum06"
```

**Gambar 1.** Pada bagian ini ditunjukkan menghubungkan *gdrive* dan memanggil *dataset*.

Selanjutnya mengimpor library seperti *pandas* dan *numpy* yang digunakan untuk pengolahan data, *matplotlib* dan *seaborn* digunakan untuk visualisasi, sementara *sklearn* digunakan untuk proses pembelajaran mesin, termasuk pembagian data, standarisasi, dan pembuatan model SVM seperti yang ditunjukkan pada Gambar 2.

## ▼ Import Library

```
[60]  
✓ Os  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

**Gambar 2.** Pada bagian ini ditunjukkan untuk mengimpor library.

Perintah `df = pd.read_csv(path + '/Data/loan.csv')` digunakan untuk membaca dataset dari folder Drive. Perintah `df.head()` digunakan untuk menampilkan lima baris pertama dari data sebagai contoh isi dataset, agar bisa dipastikan bahwa file sudah terbaca dengan benar. Perintah `df.info()` menampilkan informasi tipe data dari setiap kolom serta jumlah data yang tidak bernilai kosong. Ini membantu memastikan apakah semua kolom berisi data yang lengkap dan apakah ada kolom yang perlu diubah tipe datanya. Perintah `df.describe()` menampilkan statistika deskriptif dari kolom numerik, seperti nilai rata-rata (*mean*), nilai maksimum dan minimum, serta standar deviasi. Perintah `df.isnull().sum()` digunakan untuk mengecek jumlah nilai kosong (*missing values*) di setiap kolom. Hasilnya menampilkan angka nol di semua kolom, artinya dataset *loan.csv* bersih dan tidak memiliki data yang hilang, sehingga bisa langsung digunakan untuk tahap preprocessing dan pelatihan model tanpa perlu melakukan proses pembersihan data tambahan seperti yang ditunjukkan pada Gambar 3 dan Gambar 4.

[61]  
✓ Os

```
import pandas as pd

# membaca file csv menggunakan pandas
df = pd.read_csv(path + '/Data/loan.csv')

# cetak header data (5 baris data) dari file
df.head()
```

|   | age | gender | occupation | education_level | marital_status | income | credit_score | loan_status |
|---|-----|--------|------------|-----------------|----------------|--------|--------------|-------------|
| 0 | 32  | Male   | Engineer   | Bachelor's      | Married        | 85000  | 720          | Approved    |
| 1 | 45  | Female | Teacher    | Master's        | Single         | 62000  | 680          | Approved    |
| 2 | 28  | Male   | Student    | High School     | Single         | 25000  | 590          | Denied      |
| 3 | 51  | Female | Manager    | Bachelor's      | Married        | 105000 | 780          | Approved    |
| 4 | 36  | Male   | Accountant | Bachelor's      | Married        | 75000  | 710          | Approved    |

Next steps: [Generate code with df](#) [New interactive sheet](#)

[62]  
✓ Os

```
# Menampilkan informasi detail
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61 entries, 0 to 60
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    61 non-null    int64
1   gender                 61 non-null    object
2   occupation             61 non-null    object
3   education_level        61 non-null    object
4   marital_status         61 non-null    object
5   income                 61 non-null    int64
6   credit_score           61 non-null    int64
7   loan_status            61 non-null    object
dtypes: int64(3), object(5)
memory usage: 3.9+ KB
```

**Gambar 3.** Pada bagian ini ditunjukkan untuk membaca file csv, menampilkan 5 baris dan cek informasi data.

[63]

✓ Os

```
# Menampilkan statistika deskriptif dari dataset  
df.describe()
```

|              | age       | income        | credit_score |
|--------------|-----------|---------------|--------------|
| <b>count</b> | 61.000000 | 61.000000     | 61.000000    |
| <b>mean</b>  | 37.081967 | 78983.606557  | 709.836066   |
| <b>std</b>   | 8.424755  | 33772.025802  | 72.674888    |
| <b>min</b>   | 24.000000 | 25000.000000  | 560.000000   |
| <b>25%</b>   | 30.000000 | 52000.000000  | 650.000000   |
| <b>50%</b>   | 36.000000 | 78000.000000  | 720.000000   |
| <b>75%</b>   | 43.000000 | 98000.000000  | 770.000000   |
| <b>max</b>   | 55.000000 | 180000.000000 | 830.000000   |



[64]

✓ Os

```
df.isnull().sum()
```

|                        |   |
|------------------------|---|
|                        | 0 |
| <b>age</b>             | 0 |
| <b>gender</b>          | 0 |
| <b>occupation</b>      | 0 |
| <b>education_level</b> | 0 |
| <b>marital_status</b>  | 0 |
| <b>income</b>          | 0 |
| <b>credit_score</b>    | 0 |
| <b>loan_status</b>     | 0 |

**dtype:** int64

**Gambar 4.** Pada bagian ini ditunjukkan untuk menampilkan statistika deskriptif dan cek nilai kosong.

Selanjutnya,  $X = df.drop('loan\_status', axis=1)$  yang berarti semua kolom selain *loan\_status* digunakan sebagai variabel *independen* (fitur), sedangkan  $y = df['loan\_status']$  adalah variabel target yang akan diprediksi. Output menampilkan ukuran matriks X dan y, yaitu (jumlah data dan jumlah fitur) serta (jumlah data) seperti yang ditunjukkan pada gambar 5.

## ✓ Menentukan Fitur dan Target

```
[65]
✓ Os
X = df.drop('loan_status', axis=1)
y = df['loan_status']
print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (61, 7)
y shape: (61,)
```

**Gambar 5.** Pada bagian ini ditunjukkan untuk menentukan fitur (X) dan target (y).

Kemudian pembagian data yang dilakukan menggunakan *train\_test\_split()* dari *sklearn.model\_selection*, di mana 80% digunakan untuk data latih dan 20% untuk data uji. Output menunjukkan jumlah masing-masing data training dan testing seperti yang ditunjukkan pada Gambar 6.

## ✓ Membagi Data Training dan Testing

```
[66]
✓ Os
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Jumlah data training:", len(X_train))
print("Jumlah data testing:", len(X_test))

Jumlah data training: 48
Jumlah data testing: 13
```

**Gambar 6.** Pada bagian ini ditunjukkan pembagian data training dan data testing.

Selanjutnya dilakukan preprocessing dengan melakukan standarisasi menggunakan *ColumnTransformer*, yang menggabungkan dua langkah yaitu *StandardScaler* untuk data numerik agar semua fitur berada dalam skala yang sama, dan *OneHotEncoder* untuk data

kategorikal agar bisa diolah oleh algoritma. Output dari proses ini adalah data matriks hasil transformasi seperti yang ditunjukkan pada Gambar 7 dan Gambar 8.

## ▼ Melakukan Standarisasi

```
[67] ✓ Os  from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

categorical_features = ['gender', 'occupation', 'education_level', 'marital_status']
numerical_features = ['age', 'income', 'credit_score']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough'
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Contoh hasil preprocessing dan scaling:\n", X_train_processed[:5])
```

**Gambar 7.** Pada bagian ini ditunjukkan kode standarisasi.

Contoh hasil preprocessing dan scaling:



```
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 35 stored elements and shape (5, 47)>
  Coords      Values
(0, 0)      1.6745762293788606
(0, 1)      0.7210810703432784
(0, 2)      0.9381095179686104
(0, 3)      1.0
(0, 19)     1.0
(0, 41)     1.0
(0, 45)     1.0
(1, 0)      0.8049354054665001
(1, 1)      0.13194249372238723
(1, 2)      0.3734793378746509
(1, 3)      1.0
(1, 20)     1.0
(1, 44)     1.0
(1, 45)     1.0
(2, 0)      0.43223219521834555
(2, 1)      1.3102196469641696
(2, 2)      1.22042460801559
(2, 3)      1.0
(2, 22)     1.0
(2, 42)     1.0
(2, 45)     1.0
(3, 0)      -0.06470541844586046
(3, 1)      0.3381409955396992
(3, 2)      0.5146368828981407
(3, 4)      1.0
(3, 17)     1.0
(3, 44)     1.0
(3, 45)     1.0
(4, 0)      0.5564665986343971
(4, 1)      1.162935002808947
(4, 2)      1.0792670629921002
(4, 4)      1.0
(4, 18)     1.0
(4, 42)     1.0
(4, 45)     1.0
```

**Gambar 8.** Pada bagian ini ditunjukkan output standarisasi.

Setelah preprocessing, model SVM dibuat dengan `model = SVC(kernel='linear', random_state=42)` dan dilatih menggunakan `fit(X_train_processed, y_train)`. Kernel linear dipilih karena data tergolong sederhana dan tidak membutuhkan transformasi non-linear. Setelah model dilatih, dilakukan prediksi dengan `y_pred = model.predict(X_test_processed)`,



lalu hasilnya dievaluasi menggunakan *accuracy\_score*, *classification\_report*, dan *confusion\_matrix* seperti yang ditunjukkan pada Gambar 9.

## ▼ Membuat dan Melatih Model SVM

[68]  
✓ Os

```
model = SVC(kernel='linear', random_state=42)
model.fit(X_train_processed, y_train)
print("Model SVM berhasil dilatih.")
```

Model SVM berhasil dilatih.

## ▼ Prediksi dan Evaluasi Model

[69]  
✓ Os

```
y_pred = model.predict(X_test_processed)

print("Akurasi:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Akurasi: 0.9230769230769231

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Approved     | 1.00      | 0.89   | 0.94     | 9       |
| Denied       | 0.80      | 1.00   | 0.89     | 4       |
| accuracy     |           |        | 0.92     | 13      |
| macro avg    | 0.90      | 0.94   | 0.92     | 13      |
| weighted avg | 0.94      | 0.92   | 0.93     | 13      |

**Gambar 9.** Pada bagian ini ditunjukkan melatih dan evaluasi model.

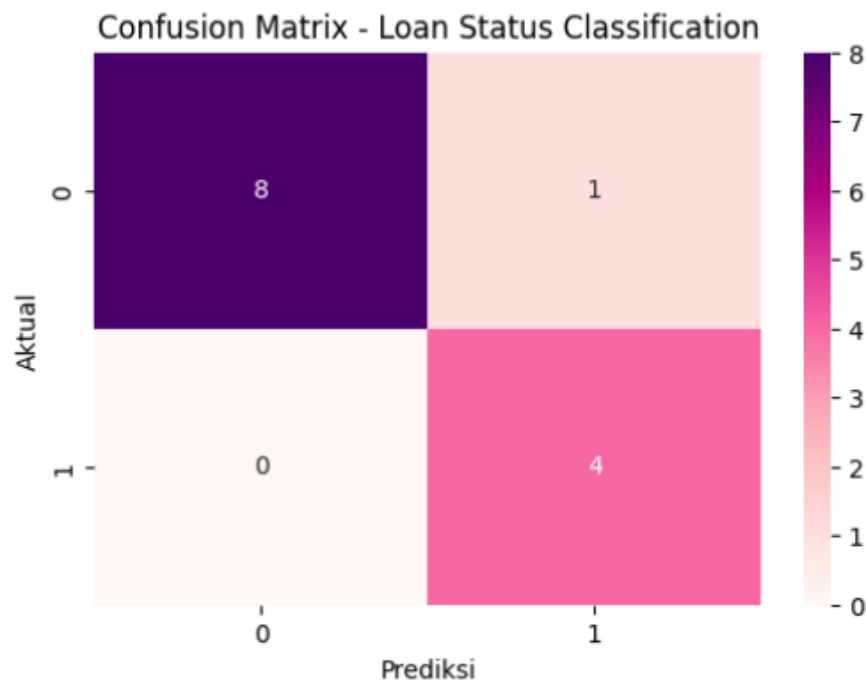
Hasil evaluasi menunjukkan akurasi model cukup tinggi, yang berarti model mampu memprediksi status pinjaman dengan baik. *Confusion matrix* divisualisasikan menggunakan *heatmap* dengan warna RdPu (ungu muda ke pink) agar terlihat jelas perbandingan antara hasil prediksi dan kondisi aktual seperti yang ditunjukkan pada Gambar 10.

## ✓ Confusion Matrix

```
[ ] print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='RdPu')
    plt.title('Confusion Matrix - Loan Status Classification')
    plt.xlabel('Prediksi')
    plt.ylabel('Aktual')
    plt.show()
```

Confusion Matrix:

```
[[8 1]
 [0 4]]
```



**Gambar 10.** Pada bagian ini ditunjukkan visualisasi confusion matrix.

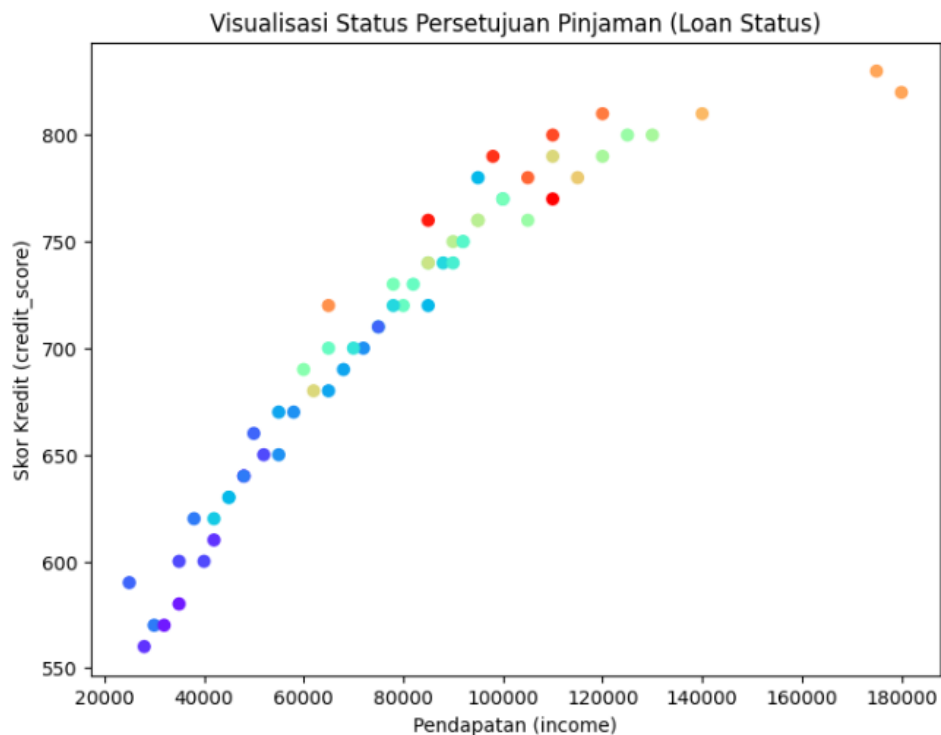
Visualisasi tambahan dibuat menggunakan *plt.scatter()* untuk menampilkan hubungan antara pendapatan, skor kredit, dan usia. Warna titik diatur dengan *cmap='rainbow'* untuk menunjukkan gradasi usia. Grafik ini memperlihatkan bahwa semakin tinggi pendapatan dan skor kredit, semakin besar kemungkinan pinjaman disetujui seperti yang ditunjukkan pada Gambar 11.

[ ]

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(df['income'], df['credit_score'], c=df['age'], cmap='rainbow')
plt.xlabel('Pendapatan (income)')
plt.ylabel('Skor Kredit (credit_score)')
plt.title('Visualisasi Status Persetujuan Pinjaman (Loan Status)')
plt.show()
```

[ ]



**Gambar 11.** Pada bagian ini ditunjukkan visualisasi 2D.

Terakhir, dibuat visualisasi 3D menggunakan *Axes3D*. Kolom *loan\_status* diubah menjadi angka dengan *LabelEncoder()*, lalu ditampilkan dalam grafik tiga dimensi dengan sumbu X (pendapatan), Y (skor kredit), dan Z (usia). Warna titik ditentukan dengan *colormap coolwarm*, sehingga titik berwarna merah mewakili status “Disetujui” dan titik biru mewakili status “Ditolak” seperti yang ditunjukkan pada Gambar 12 dan Gambar 13.

```

from sklearn.preprocessing import LabelEncoder
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import numpy as np

le = LabelEncoder()
df['LoanStatusEncoded'] = le.fit_transform(df['loan_status'])

x_col = 'income'
y_col = 'credit_score'
z_col = 'age'

colormap = plt.get_cmap('coolwarm')
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
colors = colormap(df['LoanStatusEncoded'] / df['LoanStatusEncoded'].max())
scatter = ax.scatter(
    df[x_col],
    df[y_col],
    df[z_col],
    c=colors,
    s=60,
    edgecolor='k',
    alpha=0.9
)

ax.set_xlabel('Pendapatan (Income)')
ax.set_ylabel('Skor Kredit (Credit Score)')
ax.set_zlabel('Usia (Age)')
ax.set_title('3D Visualisasi Klasifikasi Status Pinjaman (Loan Dataset)', fontsize=13)

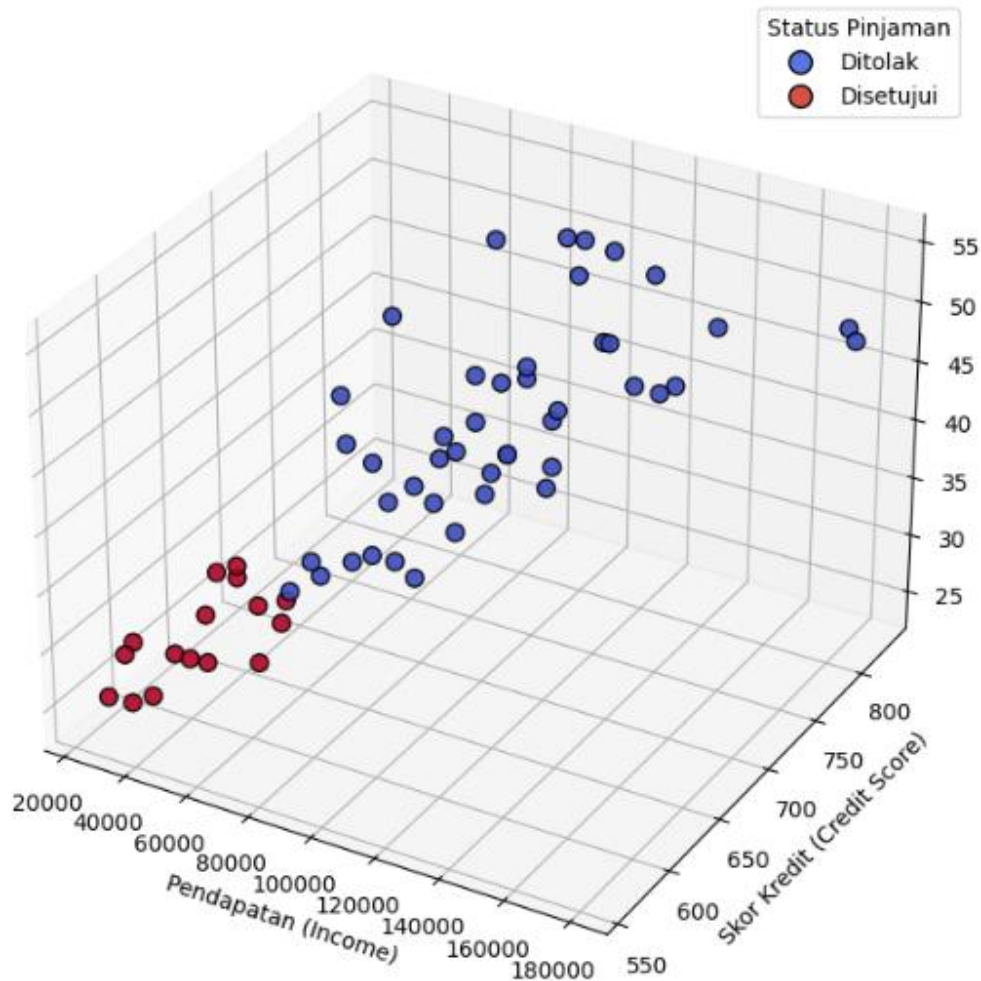
from matplotlib.lines import Line2D
custom_lines = [
    Line2D([0], [0], marker='o', color='w', label='Ditolak',
            markerfacecolor=colormap(0.1), markersize=10, markeredgecolor='k'),
    Line2D([0], [0], marker='o', color='w', label='Disetujui',
            markerfacecolor=colormap(0.9), markersize=10, markeredgecolor='k')
]
ax.legend(handles=custom_lines, title='Status Pinjaman')

plt.show()

```

**Gambar 12.** Pada bagian ini ditunjukkan kode visualisasi 3D.

### 3D Visualisasi Klasifikasi Status Pinjaman (Loan Dataset)



**Gambar 13.** Pada bagian ini ditunjukkan hasil visualisasi 3D.

### 3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi, model SVM mampu mengenali pola dengan cukup baik antara data keuangan seseorang dan status pinjaman yang diterimanya. Model menunjukkan tingkat akurasi yang tinggi dalam memprediksi kategori pinjaman, yang berarti SVM efektif dalam kasus klasifikasi seperti ini. Dari grafik 2D terlihat hubungan yang cukup kuat antara *income* dan *credit\_score*, di mana individu dengan nilai tinggi pada kedua variabel ini lebih sering masuk kategori “Disetujui”. Visualisasi 3D memperkuat hasil tersebut dengan menunjukkan distribusi data di ruang tiga dimensi, di mana kelompok “Disetujui” dan “Ditolak” terlihat terpisah cukup jelas.

Kesimpulan praktikum mandiri ini berhasil menunjukkan bagaimana algoritma *Support Vector Machine* (SVM) sangat efektif untuk melakukan klasifikasi status pinjaman berdasarkan data keuangan dan demografis. Proses preprocessing seperti standarisasi dan encoding terbukti penting agar data dapat diproses dengan benar oleh model. Selain itu, visualisasi 2D dan 3D

membantu memperjelas hubungan antar variabel dan memberikan gambaran yang lebih intuitif tentang pola data. Secara keseluruhan, model SVM berhasil diterapkan dengan baik untuk kasus prediksi status pinjaman menggunakan dataset *Simple Loan Classification* dari Kaggle.

#### **4. Link Dataset Kaggle**

<https://www.kaggle.com/datasets/sujithmandala/simple-loan-classification-dataset>

#### **5. Link GitHub (Praktikum dan Tugas Mandiri 06)**

[https://github.com/revani18/ML\\_2025\\_Revani\\_3AI01/tree/73b1c05cea5a6cb0a8cd1dd8ec93265b1ff0f4e6/Praktikum06](https://github.com/revani18/ML_2025_Revani_3AI01/tree/73b1c05cea5a6cb0a8cd1dd8ec93265b1ff0f4e6/Praktikum06)